

嵌入式作业 1: Python / C# 性能对比与 Python 优化

作业 1.1: Python 与 C# 运行时间对比

一、题目

用 Python 和 C# 各实现一段循环 100000 次的 $\text{sqrt} + \text{pow}$ 运算，测量耗时并分析。

二、测试环境

- 机器: MacBook Air (Apple M4, arm64), macOS 26.2
- Python 3.9.6 (CPython), 计时用 `datetime.datetime.now()`
- C#: Mono 6.14.1, `mcs -optimize` 编译, 计时用 `Stopwatch`
- 两边都重复 10 轮取平均, 减小偶然误差

源码: `python_benchmark.py`、`CSharpBenchmark/Program.cs`

三、实测结果

语言	平均耗时 (10 万次循环)	相对倍率
C#	约 1.15 ms	1.0×
Python	约 9.33 ms	约 8.1×

C# 比 Python 快约 8 倍。

四、结果分析

差距主要来自三点:

1. 执行方式: C# 经 JIT 编译为本地机器码, 循环体几乎就是几条本地指令; Python 是字节码解释执行, 每次迭代都要走解释器分派和 C API 调用。
2. 数据类型: C# 的 `int / double` 是栈上值类型, 零分配; Python 的 `int / float` 是堆上对象, 每次运算都涉及装箱和引用计数。
3. 循环开销: `for (int i = 0; ...) in C# 里是 inc + cmp + jl, 几乎免费; for number in range(...) 在 Python 里要反复调用迭代器的 __next__。`

五、结论

数值密集型计算场景下, 编译型语言 (C#) 相比解释型语言 (Python) 有明显优势。Python 适合做原型和上层逻辑, 性能关键路径更适合用 C# / C / C++ 这类编译到本地代码的语言。

作业 1.2: Python 代码优化

一、目标

参考 [riverzhou/PythonSpeedTest](#) 的方案, 对作业 1.1 中的 Python 代码进行多种方式的加速, 并以表格形式给出对比结果。

为了让差距更明显, 把数据规模从 10 万扩大到 100 万, 运算保持为 $\text{sqrt}(i) + i**2$ 。

二、优化方案

参照开源项目, 本次实现了从「纯 Python 循环」到「OpenMP 并行 C 模块」共 8 种实现:

Name	Information	优化思路
calc_py	Standard Python Loop	基线，等价于作业 1.1 的写法 (math + range)
calc_np	NumPy Vectorized	用 NumPy 数组 + 广播代替 for 循环 (矩阵化)
calc_np_opt	NumPy Optimized (i*i)	在向量化基础上把 np.power(v, 2) 换成 v*v
calc_jit	Numba JIT	@njit 装饰器，运行时 JIT 编译为本地代码
calc_aot	Numba AOT	numba.pycc 预编译为扩展模块 (.so)
calc_cy	Cython Module	用 Cython 重写循环并编译为扩展模块
calc_c	C Module (clang -O3)	自己写 C 代码编译成动态库，ctypes 调用
calc_omp	C Module + OpenMP	在 C 实现上加 #pragma omp parallel for 多线程并行

源码与构建脚本均在 1.2_optimized/ 目录下，build.sh 会一次性编译出所有原生扩展，SpeedTest.py 完成正确性校验和 5×5 计时。

三、测试环境（实验机）

题目提示「在 Windows 上实验」，本机为 macOS (Apple M4)，把同一套思路移植到本地：原项目的 .dll 替换为 .dylib，OpenMP 通过 brew 的 libomp 提供，其它 (NumPy / Numba / Cython) 一致。

项目	取值
OS	macOS 26.2 / Darwin 25.2.0 (arm64)
CPU	Apple M4, 10 线程
Python	3.9.7 (conda)
库	NumPy 2.0.2 / Numba 0.60.0 / Cython 3.2.4
编译器	Apple clang 17 (C), libomp (OpenMP)

四、测试结果

数据规模 1,000,000，类型 numpy.int32，每项跑 5 轮 × 每轮 5 次（与开源项目一致）：

Test Result

Name	Information	1	2	3	4	5	Avg	Faster
calc_py	Standard Python Loop	0.857	0.850	0.849	0.851	0.940	0.870	1.000
calc_np	NumPy Vectorized	0.040	0.034	0.036	0.034	0.036	0.036	24.297
calc_np_opt	NumPy Optimized (i*i)	0.020	0.011	0.010	0.009	0.009	0.012	73.253
calc_jit	Numba JIT	0.004	0.003	0.004	0.003	0.003	0.003	255.008
calc_aot	Numba AOT	0.006	0.005	0.005	0.005	0.005	0.005	169.577
calc_cy	Cython Module	0.005	0.004	0.005	0.004	0.004	0.005	192.657
calc_c	C Module (clang -O3)	0.003	0.003	0.003	0.003	0.006	0.004	232.779
calc_omp	C Module + OpenMP	0.002	0.002	0.002	0.004	0.005	0.003	302.218

时间单位：秒。Faster 列为相对 calc_py 的加速比。

五、结果分析

可以分成四个层级看：

1. 从纯 Python 到 NumPy (24×) calc_np 把循环交给 NumPy 在 C 层面用 SIMD/连续内存做批量运算，单次就完成了 100 万次 sqrt + 平方。开销从「100 万次解释器调度」降到「一次函数调用 + 一次内存遍历」。
2. 微优化 i*i (73×) np.power(v, 2) 走的是通用指数实现 (log/exp)，而 v*v 是一次乘法。同样是 NumPy 向量化，仅这一处就再快 3 倍，说明热点函数本身的实现成本不可忽视。

3. **JIT / AOT / Cython / C (170×-255×)** 这一层都把循环编译成了本地代码，差距主要来自编译器/运行时的细节：
- **calc_jit**: Numba JIT 在第一次调用时已经触发并由 `cache=True` 命中缓存，后续走的是 LLVM 优化过的本地代码，最接近手写 C；
 - **calc_aot**: Numba AOT 编译路径与 JIT 类似但少了一些运行时调度优化，略慢于 JIT；
 - **calc_cy**: Cython 通过类型注解直接调用 `libc.math` 的 `sqrt`，性能稳定；
 - **calc_c**: `clang -O3 -ffast-math`，向量化得很彻底，单线程已经很难再快；
 - 这一层之间的差距主要是测量噪声，量级一致。
4. **OpenMP 并行 (302×)** `calc_omp` 在 C 实现的循环上加了 `#pragma omp parallel for`，把任务切到 10 个线程上同时算。M4 上能再多拿 ~1.3 倍，受限于此「一遍读、一遍写」的工作几乎是内存带宽 bound，加再多线程也提升有限。

六、结论

- 把作业 1.1 的写法从「Python for 循环」改成「NumPy 向量化」是性价比最高的一步，几行改动换来 24× 提升，再把 `power(v, 2)` 换成 `v*v` 又能多 3 倍。
- 进一步追求性能时，**Numba JIT** 是最划算的方案：只要加一个 `@jit` 装饰器，就能拿到接近手写 C 的速度 (255×)，不需要写 C/构建系统。
- 当循环本身已经是瓶颈、并且数据规模足够大时，**OpenMP** 能再吃掉一点 CPU 多核红利 (302×)，但收益取决于内存带宽。
- 与作业 1.1 的 Python (9.33 ms / 10 万次) 对照来看：在 100 万规模下，最快的 OpenMP 版本只用约 3 ms，已经显著快于原版 C# 实现，说明 Python 的「慢」并不是语言本身的硬伤，而是循环 + 对象模型的代价；只要把热点交给本地代码或矩阵运算，性能差距是可以填平甚至反超的。

七、复现步骤

```
# 1) 准备依赖
/opt/miniconda3/bin/pip install numpy numba cython
brew install libomp          # 若已安装可跳过

# 2) 编译所有原生模块
cd 1.2_optimized
bash build.sh

# 3) 跑测试，结果会写入 1.2_optimized/Result.md
/opt/miniconda3/bin/python SpeedTest.py
```

嵌入式作业 2: Replicable Deadlock

一、实验目标

在不同平台上重复运行题目给出的 Java 死锁程序，比较死锁复现情况。源码位于 作业 2_Deadlock/，测试脚本为 `test_deadlock.py`。

程序中有两把对象锁：A 和 B。主线程进入 `a.methodA(b)` 时先持有 A 锁，再请求 B 锁；子线程进入 `b.methodB(a)` 时先持有 B 锁，再请求 A 锁。如果两个线程刚好同时持有第一把锁，就会形成循环等待。

二、实验方法

每个平台重复运行 100 次。单次运行超过 3 秒未退出，记为死锁；正常打印 `Inside A.last()` 和 `Inside B.last()` 并退出，记为正常。

```
python 作业 2_Deadlock/test_deadlock.py --runs 100 --timeout 3 --log run.log
```

Linux、Windows、macOS 使用 GitHub Actions 测试；本机 MacBook Air 作为 Laptop / PC 测试；移动端使用 Android 手机运行同一测试脚本。

三、三组对比结果

1. Linux vs. Windows

平台	测试环境	正常退出	死锁次数	死锁率
Linux	GitHub Actions ubuntu-latest, Temurin OpenJDK 21	100	0	0.0%
Windows	GitHub Actions windows-latest, Temurin OpenJDK 21	98	2	2.0%

Windows 上出现了 2 次死锁，Linux 上本轮没有复现。说明同一段程序在不同操作系统调度下，死锁复现概率不同。

2. Desktop vs. Laptop

设备类型	测试环境	正常退出	死锁次数	死锁率
Desktop / Cloud runner	GitHub Actions macos-latest, Temurin OpenJDK 21	99	1	1.0%
Laptop	MacBook Air, Apple M4, OpenJDK 25.0.2	100	0	0.0%

云端 macOS runner 出现 1 次死锁，本机笔记本未复现。两者差异可能来自 CPU 性能、虚拟化环境、JVM 版本和线程调度策略。

3. PC vs. Mobile Device

设备类型	测试环境	正常退出	死锁次数	死锁率
PC	MacBook Air, Apple M4, OpenJDK 25.0.2	100	0	0.0%
Mobile device	Android + Termux + OpenJDK	96	4	4.0%

移动端出现 4 次死锁，死锁率高于本机 PC。说明移动设备的 CPU 性能、系统调度和 Java 运行环境会明显影响该程序的复现概率。

四、结果分析

该程序满足死锁的四个必要条件：互斥、占有并等待、不可抢占、循环等待。但题目原代码只用 `while (count-- > 0)`；进行忙等，没有保证两个线程一定同时拿到第一把锁。

因此，它不是确定性死锁程序，而是一个依赖调度时机的低概率死锁程序。Windows、云端 macOS 和 Android 手机在 100 次内低概率复现，Ubuntu 和本机 MacBook 本轮未复现。

五、结论

本实验说明：死锁风险由代码结构决定，但能否复现受操作系统、JVM、CPU 性能和线程调度影响。同一份 Java 程序在不同平台上的复现率不同，不能简单认为运行一次正常就没有死锁问题。

嵌入式作业 3：实时系统调度分析

一、任务参数

Task	Cx	Tx	Dx
τ_1	2	10	10
τ_2	4	18	18
τ_3	5	12	12

系统为单核心、可抢占调度，且所有任务在系统开始运行时同时就绪。

二、总利用率

$$\begin{aligned}U &= C_1/T_1 + C_2/T_2 + C_3/T_3 \\&= 2/10 + 4/18 + 5/12 \\&= 151/180 \\&= 0.839\end{aligned}$$

所以系统总利用率为 **0.839**。

三、Fixed-Priority Scheduling

固定优先级按 Rate Monotonic 顺序，即周期越短优先级越高：

$$\tau_1 > \tau_3 > \tau_2$$

调度过程前段如下：

时间区间	执行任务
0 - 2	τ_1 第 1 次 release
2 - 7	τ_3 第 1 次 release
7 - 10	τ_2 第 1 次 release
10 - 12	τ_1 第 2 次 release
12 - 17	τ_3 第 2 次 release
17 - 18	τ_2 第 1 次 release 剩余部分

因此：

τ_1 第 2 次 release 的完成时间 = 12

τ_3 第 1 次 release 的完成时间 = 7

四、Earliest Deadline First

EDF 每次选择绝对截止时间最早的任务执行。

调度过程前段如下：

时间区间	执行任务
0 - 2	τ_1 第 1 次 release, deadline = 10
2 - 7	τ_3 第 1 次 release, deadline = 12
7 - 11	τ_2 第 1 次 release, deadline = 18
11 - 13	τ_1 第 2 次 release, deadline = 20

因此：

τ_1 第 2 次 release 的完成时间 = 13

τ_3 第 1 次 release 的完成时间 = 7

五、Utilization Bound 判断

1. Fixed-Priority Scheduling

3 个周期任务的 Rate Monotonic 利用率上界为：

$$\begin{aligned}U_{RM} &= n(2^{(1/n)} - 1) \\&= 3(2^{(1/3)} - 1) \\&= 0.780\end{aligned}$$

实际利用率：

$$U = 0.839 > 0.780$$

所以该任务集 不满足 RM 利用率上界的充分条件，不能仅凭 utilization bound 判定为 Fixed-Priority Scheduling 下可调度。

2. Earliest Deadline First

对于单核心、可抢占、相对截止时间等于周期的周期任务，EDF 的利用率上界为：

$$U_{EDF} = 1$$

实际利用率：

$$U = 0.839 \leq 1$$

所以该任务集在 EDF 下可调度。

嵌入式作业 4：MSRP 阻塞分析

一、任务与资源

题目中 4 个任务都访问共享资源 R1，处理器亲和性如下：

Task	Priority	Resource	Processor
T1	1	R1	P1
T2	2	R1	P2
T3	3	R1	P1
T4	4	R1	P2

按题图理解，优先级数值越大，优先级越高。因此在 P1 上 $T3 > T1$ ，在 P2 上 $T4 > T2$ 。

二、补全运行示例图

根据图中的资源访问关系，缺失的阻塞段可按下面方式标注：

任务	Blocking effect
T1	Indirect spin delay
T2	Direct spin delay, Indirect spin delay

任务	Blocking effect
T3	Arrival blocking, Direct spin delay
T4	Arrival blocking, Direct spin delay

说明：

- **Direct spin delay**: 任务自己请求 R1，但 R1 正被另一个处理器上的任务占用，只能自旋等待。
- **Indirect spin delay**: 本处理器上更高优先级任务正在自旋，占用处理器，使低优先级任务间接受到延迟。
- **Arrival blocking**: 任务到达时，本处理器上低优先级任务正在不可抢占地访问全局资源，导致新到达任务暂时不能运行。

三、T3 的阻塞时间

从图中读出 T3 的阻塞包括两段：

阻塞类型	时间区间	时长
Arrival blocking	1 - 4	3
Direct spin delay	5 - 6	1

因此 T3 在该运行示例中的阻塞时间总和为：

$$B_{T3} = 3 + 1 = 4$$

四、是否为 T3 的最差情况

该阻塞时间总和 不是 T3 的最差情况。

原因是题图只给出了一种可能的资源访问时刻。理论上，任务可以在其运行过程中的任意时刻请求 R1。如果 T3 请求 R1 的时刻刚好落在 P2 上某个任务刚进入 R1 临界区之后，T3 的 direct spin delay 可能比图中的 1 个时间单位更长。因此，图中的 $B_{T3} = 4$ 只是该示例调度下的阻塞时间，不一定是最好阻塞时间。